Treap[1] is just the *portmanteau*[2] of tree and heap: binary search trees, in fact, offer the best average performance across all standard operations: `insert`, `remove` and `search` (and also `min` and `max`).

Heaps, on the other hand, allow to efficiently keep track of priorities using a tree-like structure. Since binary heaps are also binary trees, the two structures seem compatible, we only need to find a way to make them co-exist in the same structure and we could get the best of both.

It's easier said than done, however! If we have a set of unidimensional data, we can't enforce *BST*'s and heap's invariants at the same time:

- Either we add a "horizontal" constrain (given a node `N`, with two children `L`, its left child, and `R`, its right child, then all keys in the left subtree – rooted at `L` - must be smaller than `N`'s key, and all keys in the right subtree – rooted at `R` - must be larger than `N`'s key);
- Or we add a "vertical" constraint (the key in the root of any subtree must be the smallest of the subtree).

Anyway, we are in luck, since each of our entries have two values: its name, and the stock inventory. The idea, therefore, is to enforce BST's constraints on the names, and heap's constraint on the quantities, obtaining something of the like of figure 3.2.

In this example, product names are treated as the keys of a binary search tree, and so they define a total ordering (from left to right in the figure).

The inventory quantities, instead, are treated as priorities of a heap, and so they define a partial ordering from top to bottom. For priorities, like all heaps we have a partial ordering, meaning that only nodes on the same path from root to leaves are ordered